

1. Gestión de Bases de Datos

Antes de poder crear tablas (como usuarios o productos) o buscar información, necesitamos un "contenedor" principal para nuestro proyecto. Ese contenedor es la Base de Datos en sí. Un mismo servidor MySQL puede alojar múltiples bases de datos para diferentes proyectos.

Crear una nueva Base de Datos

El comando `CREATE DATABASE` se utiliza para instanciar una nueva base de datos vacía. Es recomendable verificar si ya existe una con el mismo nombre para evitar errores usando la cláusula `IF NOT EXISTS`.

```
-- Crear la base de datos solo si no existe previamente CREATE DATABASE IF NOT EXISTS tienda_online; -- Listar todas las bases de datos disponibles en el servidor SHOW DATABASES;
```

Seleccionar la Base de Datos Activa

Una vez creada, MySQL no la selecciona automáticamente. Debemos indicarle explícitamente al sistema que todas las operaciones siguientes (crear tablas, insertar datos, buscar) se realizarán sobre esa base de datos específica utilizando el comando `USE`.

```
-- Seleccionar la base de datos para empezar a trabajar en ella USE tienda_online;
```

Eliminar una Base de Datos

Si un proyecto ya no es necesario o deseas reiniciar desde cero, puedes eliminar la base de datos completa. Esta operación es destructiva y eliminará permanentemente la base de datos y **todas las tablas y registros** que contenga.

```
-- Eliminar la base de datos por completo DROP DATABASE tienda_online;
```

2. La Búsqueda de Datos: La cláusula WHERE

Una vez que tienes datos en tus tablas, rara vez querrás verlos todos al mismo tiempo. Para buscar registros específicos que cumplan con ciertos criterios, utilizamos la cláusula WHERE dentro de una sentencia SELECT.

Operadores de Comparación

El filtrado básico se realiza comparando los valores de las columnas con los valores que estamos buscando.

Operador	Significado	Ejemplo de uso en WHERE
=	Igual a	WHERE edad = 25
> / <	Mayor que / Menor que	WHERE precio > 100
>= / <=	Mayor o igual / Menor o igual	WHERE stock <= 10
!= o <>	Diferente de (No es igual a)	WHERE estado != 'cancelado'

Operadores Lógicos (Múltiples condiciones)

Frecuentemente, las búsquedas requieren cumplir más de una condición al mismo tiempo, o al menos una de varias opciones. Para esto usamos AND, OR y NOT.

```
-- Búsqueda con AND: Ambas condiciones DEBEN ser verdaderas -- Busca productos electrónicos que cuesten menos de 500
SELECT nombre, precio FROM productos WHERE categoria = 'Electrónica' AND precio < 500;
-- Búsqueda con OR: Al menos UNA condición debe ser verdadera -- Busca clientes de España o de México
SELECT nombre, pais FROM clientes WHERE pais = 'España' OR pais = 'México';
-- Uso de paréntesis para agrupar condiciones complejas
SELECT * FROM pedidos WHERE (estado = 'Pendiente' OR estado = 'En Proceso') AND total > 1000;
```

3. Búsqueda Avanzada: Patrones y Listas

MySQL ofrece herramientas de búsqueda que van más allá de la coincidencia exacta de texto o números, permitiéndonos buscar coincidencias parciales, rangos o listas de valores.

Buscando texto parcial con LIKE

El operador LIKE se utiliza junto con dos comodines especiales para buscar un patrón específico en una columna de texto:

- El signo de porcentaje (%) representa cero, uno o múltiples caracteres.
- El guion bajo (_) representa un único carácter exacto.

```
-- Buscar todos los nombres que COMIENCEN con la letra 'A' SELECT nombre FROM usuarios WHERE nombre LIKE 'A%'; -- Buscar todos los correos que CONTENGAN la palabra 'gmail' en cualquier parte SELECT email FROM usuarios WHERE email LIKE '%gmail%'; -- Buscar nombres que terminen con 'ez' (ej. Pérez, Gómez) SELECT apellido FROM usuarios WHERE apellido LIKE '%ez';
```

Búsqueda en Listas (IN) y Rangos (BETWEEN)

Cuando necesitas comprobar si un valor coincide con cualquiera dentro de una lista específica, usar múltiples OR puede hacer el código muy largo. IN simplifica esto. Por otro lado, BETWEEN es ideal para buscar dentro de un rango continuo de números o fechas.

```
-- Uso de IN: Buscar usuarios que tengan roles específicos -- Equivalente a: rol = 'Admin' OR rol = 'Editor' OR rol = 'Autor' SELECT nombre, rol FROM empleados WHERE rol IN ('Admin', 'Editor', 'Autor'); -- Uso de BETWEEN: Buscar productos cuyo precio esté entre 50 y 150 SELECT producto, precio FROM inventario WHERE precio BETWEEN 50 AND 150; -- BETWEEN también es excelente para rangos de fechas SELECT * FROM ventas WHERE fecha_venta BETWEEN '2026-01-01' AND '2026-12-31';
```

4. Trabajando con Datos Incompletos: Valores Nulos

En bases de datos, un valor NULL significa que el dato está "ausente" o es "desconocido". Un campo vacío ("") o un cero (0) no son lo mismo que NULL. No puedes buscar valores nulos usando el signo igual (= NULL); debes usar los operadores IS NULL o IS NOT NULL.

```
-- Buscar usuarios que NO han proporcionado un número de teléfono SELECT nombre
FROM clientes WHERE telefono IS NULL; -- Buscar usuarios que SÍ tienen una
dirección registrada SELECT nombre FROM clientes WHERE direccion IS NOT NULL;
```

5. Limitación y Paginación de Búsquedas

Si una búsqueda devuelve miles de resultados, es poco eficiente enviarlos todos de golpe a la aplicación. Las cláusulas LIMIT y OFFSET se utilizan para restringir cuántos registros se devuelven y desde dónde empezar a contar, siendo la base técnica de la "paginación" en páginas web.

Buena Práctica: Siempre ordenar antes de limitar

Si vas a limitar los resultados, siempre acompáñalo de un ORDER BY. Si no ordenas los resultados de antemano, MySQL devolverá los registros en un orden arbitrario, haciendo que tus búsquedas sean impredecibles.

```
-- Obtener los 5 productos más caros de la tienda SELECT nombre, precio FROM
productos ORDER BY precio DESC LIMIT 5; -- Ejemplo de Paginación: Saltar los
primeros 10 resultados, -- y devolver los 10 siguientes (es decir, la "Página 2")
SELECT nombre, email FROM usuarios ORDER BY id ASC LIMIT 10 OFFSET 10;
```